



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

Subject : Special Topic in Data Engineering (SECP3843)

Semester : 2/20252026

Section : 02

Task : Tutorial 4: Claude to Pipeline

Lecturer : Dr. Aryati Binti Bakri

Group : Section 2 Group 1

Prepared by:

No.	Name	Matric Number
1	Lee Yin Shen	A23CS0236
2	Joanne Ching Yin Xuan	A23CS0227
3	Evelyn Goh Yuan Qi	A23CS0222

Table of Contents

1.0 Introduction.....	3
2.0 What is AI-assisted Data Engineering?.....	4
3.0 Claude and AI Agents Used in the Tutorial.....	5
4.0 Pipeline Architecture and Implementation Overview.....	6
5.0 Use Case Diagram of Car Sales Pipeline System.....	7
6.0 Steps to Do Prompt and Coding.....	9
6.1 Step 1: AI-Assisted Dataset Understanding.....	9
6.2 Step 2: AI-Assisted Pipeline Planning.....	10
6.3 Step 3: AI-Assisted Code Generation.....	11
6.4 Step 4: AI-Assisted dbt Model Generation.....	12
6.4.1 Staging Layer.....	12
6.4.2 Mart Layer.....	13
6.4.3 Output.....	13
6.5 Step 5: AI-Assisted Debugging and Improvement.....	14
6.6 Step 6: Final Pipeline Execution.....	15
7.0 Results and Output Explanation.....	16
7.1 DuckDB Warehouse Results.....	16
7.2 CSV Output Results.....	17
7.3 Visualization Results.....	17
7.3.1 Sales by Salesperson.....	17
7.3.2 Sales by Car Make.....	18
7.3.3 Monthly Sales Trend.....	19
7.3.4 Commission Rate Distribution.....	20
7.4 Pipeline Summary Report.....	21
8.0 Future Improvements.....	22
9.0 Reflection.....	24
References.....	27

1.0 Introduction

Artificial Intelligence (AI) is increasingly being integrated into software and data engineering workflows to improve productivity, automate repetitive tasks, and support decision-making processes. Recent advances in generative AI have enabled intelligent systems to assist users in activities such as code generation, debugging, documentation, and workflow planning, thereby reducing development effort and improving operational efficiency (Dwivedi et al., 2023).

This report is prepared for Tutorial 4: Claude & Pipeline, based on the selected tutorial video entitled “*The AI Workflow for Data Engineering.*” The tutorial demonstrates how AI can be utilized to support various data engineering activities, including pipeline planning, code generation, debugging, data transformation, and workflow optimization. In the original tutorial, Claude Code is employed as an AI agent to assist in the development of a modern data pipeline consisting of data ingestion, data warehousing, transformation models, analytical data marts, and reporting outputs.

To demonstrate the concepts presented in the tutorial, a local AI-assisted data engineering pipeline was developed using a car sales dataset. The implementation adopts the same workflow principles as the original tutorial while utilizing technologies that can be executed in a local development environment. DuckDB is used as the analytical data warehouse, dbt is used to manage staging and mart transformation models, and Python is used for data ingestion, orchestration, data export, visualization generation, and pipeline execution management.

The implemented workflow begins with a raw car sales dataset, followed by AI-assisted planning and code generation, data ingestion into DuckDB, transformation through dbt models, generation of analytical outputs, and visualization of results using Matplotlib. This project demonstrates how AI can effectively support the development of end-to-end data pipelines while emphasizing the importance of human validation and review to ensure the accuracy, reliability, and quality of the final solution.

2.0 What is AI-assisted Data Engineering?

AI-assisted data engineering refers to the use of artificial intelligence (AI) technologies to support the development, management, and optimization of data engineering workflows. Traditionally, data engineers perform tasks such as data ingestion, data cleaning, transformation, validation, and storage manually. Recent advancements in large language models (LLMs) have enabled AI systems to assist in software engineering activities such as code generation, debugging, documentation, and workflow automation, improving development efficiency and reducing manual effort (Zhang et al., 2024).

The integration of AI into data engineering helps improve productivity and reduce development time. AI assistants such as Claude can understand user requirements through natural language prompts and generate Python scripts, SQL queries, and workflow recommendations. In addition, AI can assist in debugging by explaining errors and proposing possible solutions.

For example, an AI assistant can analyze a dataset and recommend the appropriate steps for data cleaning and transformation. It can also generate Python code to load data into a database, create SQL models for data transformation, and suggest data quality checks. This enables data engineers to develop pipelines more efficiently while reducing repetitive manual work.

In this project, AI was used to support multiple stages of pipeline development, including dataset analysis, pipeline planning, code generation, and troubleshooting. The AI assistant provided recommendations that helped accelerate the implementation of the car sales data pipeline using Python, DuckDB, and dbt.

Although AI can automate many technical tasks, human validation remains essential. Data engineers are responsible for reviewing AI-generated outputs, verifying pipeline logic, and ensuring that the results are accurate and meaningful. Therefore, AI-assisted data engineering should be viewed as a collaboration between AI tools and human expertise rather than a replacement for data engineers.

3.0 Claude and AI Agents Used in the Tutorial

In the selected tutorial, Claude Code was introduced as an AI agent that could support data engineering workflows through pipeline planning, code generation, debugging, and workflow improvement. This reflects the broader role of generative AI in technical work, where AI tools are increasingly being used to support coding, documentation, problem solving, and decision-making activities (Dwivedi et al., 2023). In this project, the same concept was adopted by using AI assistants to support the development of the car sales data pipeline.

Different AI tools were used by different group members, including ChatGPT, Gemini, Codex, and Claude. Although different tools were used, they were applied for the same general purpose: assisting with dataset understanding, pipeline planning, coding, debugging, and report explanation. This is consistent with recent studies on large language models in software engineering, which show that LLMs can support activities such as code generation, program repair, testing, documentation, and software maintenance (Zhang et al., 2024).

In this project, AI assistance was used to help design the pipeline structure, generate the Python orchestration script, create DuckDB and dbt transformation logic, solve technical errors, and refine the documentation. The original tutorial used Claude Code in a cloud-based workflow, while this project adapted the same AI-assisted workflow concept into a local environment using Python, DuckDB, dbt, pandas, and Matplotlib.

Although AI was used throughout the project, the generated outputs were still reviewed by the group members. The Python script, dbt models, SQL logic, generated tables, CSV outputs, charts, and terminal results were checked to ensure that they matched the project requirements. This was necessary because AI-generated outputs may still require human validation to ensure accuracy, reliability, and suitability for the intended task. Therefore, AI was used as a collaborative assistant, while final validation and decision-making remained the responsibility of the team.

4.0 Pipeline Architecture and Implementation Overview

The project pipeline was designed based on the workflow concept demonstrated in the selected tutorial. However, the implementation was adapted to a local development environment so that it could be executed without relying on a cloud warehouse. The final pipeline was developed using Python, DuckDB, dbt, pandas, and Matplotlib.

The implemented workflow can be summarized as follows:

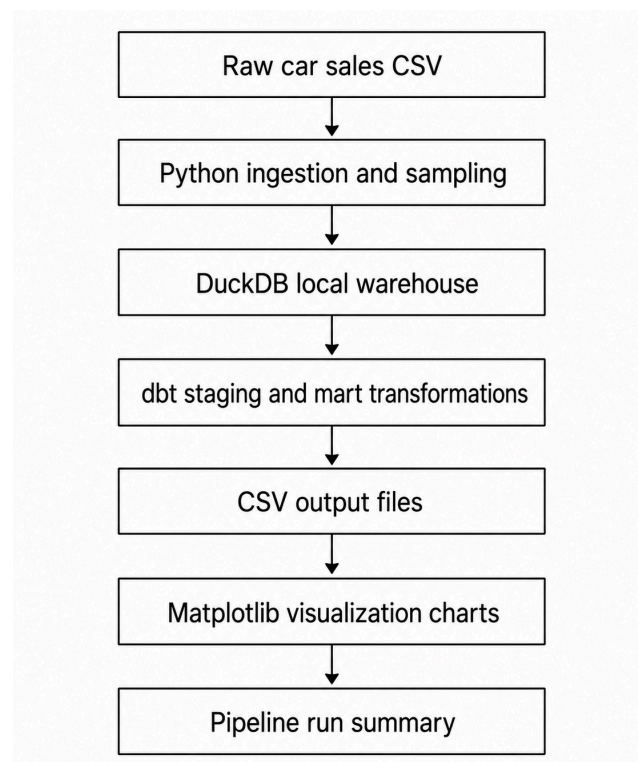


Figure 4.1: Overall Workflow of the AI-Assisted Car Sales Data Pipeline

Figure 4.1 shows the overall workflow of the implemented car sales data pipeline. The process began with the raw `car_sales_data.csv` file, which was read and sampled using Python and pandas. The sampled data was then loaded into DuckDB as the `raw_car_sales` table, where DuckDB acted as the local data warehouse.

After the raw table was created, dbt was used to manage the transformation process. The `stg_car_sales` model cleaned and standardized the data, while the mart models generated analytical outputs such as data quality results, salesperson performance, car make summaries, car model summaries, and monthly sales trends.

The transformed tables were exported as CSV files under data/processed, and Matplotlib was used to generate the visualization charts. Finally, a pipeline_run_summary.txt file was produced to summarize the pipeline status, row counts, generated outputs, and validation results. Overall, the workflow shows how Python, DuckDB, dbt, and Matplotlib were connected to form an end-to-end AI-assisted data engineering pipeline.

5.0 Use Case Diagram of Car Sales Pipeline System

Before the use case diagram was developed, the pipeline process was first identified based on the implemented Python script and dbt models. The pipeline was started by reading the raw car_sales_data.csv file using Python and pandas. A 50,000-row sample was then created for faster execution and demonstration. The sampled data was loaded into DuckDB as the raw_car_sales table, where DuckDB acted as the local data warehouse.

After the raw table was created, dbt was executed from the Python script to perform the transformation process. The stg_car_sales model was used to clean and standardize the raw data, while the mart models were used to generate the data quality summary, salesperson metrics, car make summary, car model summary, and monthly sales summary. After the dbt models were completed, the Python script exported the generated tables into CSV files, produced Matplotlib charts, and created the pipeline_run_summary.txt file.

The use case diagram in Figure 5.1 represents the interaction between the users, AI assistants, and the AI-Assisted Car Sales Pipeline System. The group members acted as the main users who provided requirements, executed the pipeline, reviewed the outputs, and prepared the report evidence. AI assistants, including ChatGPT, Gemini, Codex, and Claude, were used to support pipeline planning, code generation, debugging, and improvement. The system boundary represents the complete pipeline system that handled ingestion, warehouse loading, dbt transformation, output generation, visualization, and summary reporting.

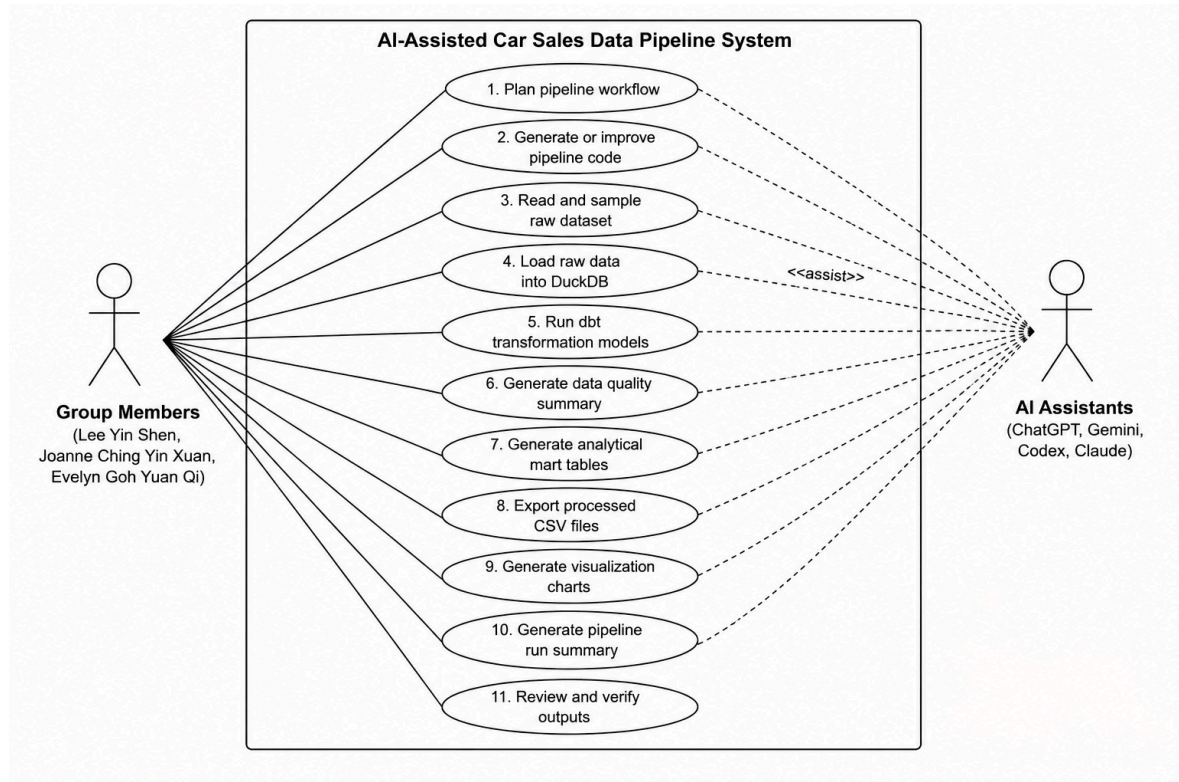


Figure 5.1: Use Case Diagram of the AI-Assisted Car Sales Pipeline System

The main use cases involved in the system include planning the pipeline workflow, generating or improving code, reading and sampling the raw dataset, loading data into DuckDB, running dbt transformation models, generating data quality and mart tables, exporting CSV outputs, generating charts, creating the pipeline run summary, and reviewing the final outputs. These use cases reflect the actual processes implemented in the script and can be traced to functions such as `load_raw_csv()`, `create_raw_table()`, `run_dbt()`, `export_csv_outputs()`, `create_charts()`, and `generate_run_summary()`.

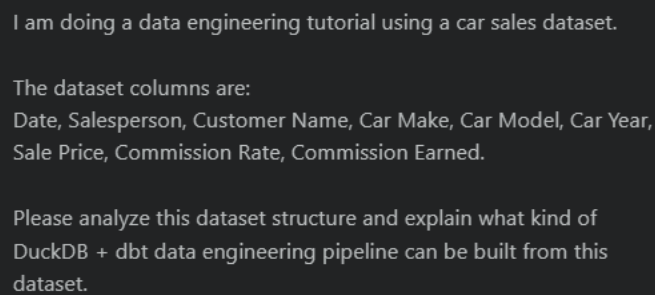
6.0 Steps to Do Prompt and Coding

This section explains how AI Assistants was used throughout the development of the AI-Assisted Car Sales Data Pipeline. AI was integrated into the workflow to support dataset understanding, pipeline planning, code generation, debugging, and code improvement.

6.1 Step 1: AI-Assisted Dataset Understanding

The first step was to understand the structure of the car sales dataset and identify a suitable data engineering workflow. The dataset contains sales transaction information including sales date, salesperson, customer details, vehicle information, sale price, commission rate, and commission. This recommendation provided a clear direction for the project before implementation began.

Prompt Used



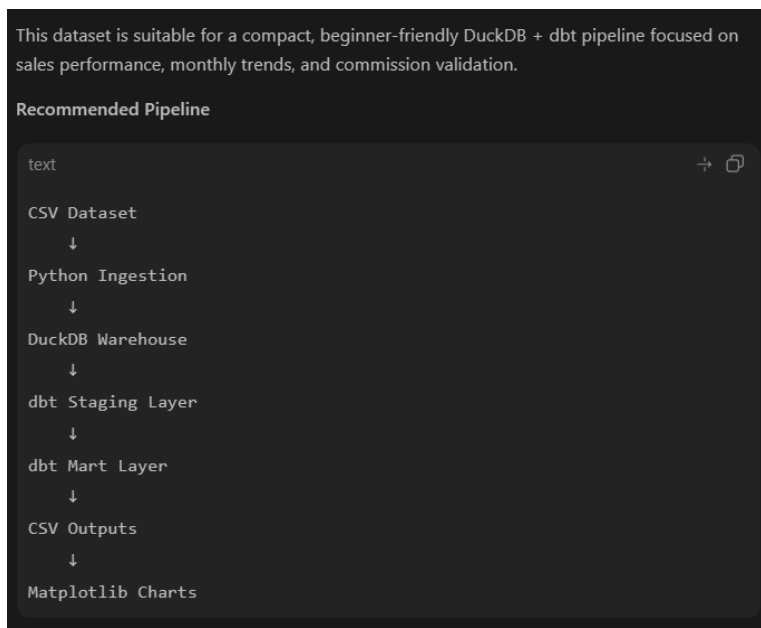
```
I am doing a data engineering tutorial using a car sales dataset.

The dataset columns are:
Date, Salesperson, Customer Name, Car Make, Car Model, Car Year,
Sale Price, Commission Rate, Commission Earned.

Please analyze this dataset structure and explain what kind of
DuckDB + dbt data engineering pipeline can be built from this
dataset.
```

Figure 6.1: Prompt for Dataset Understanding

AI Output



```
This dataset is suitable for a compact, beginner-friendly DuckDB + dbt pipeline focused on
sales performance, monthly trends, and commission validation.

Recommended Pipeline

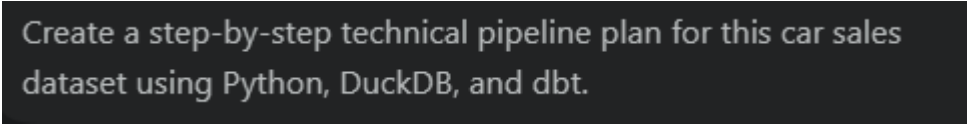
text
CSV Dataset
↓
Python Ingestion
↓
DuckDB Warehouse
↓
dbt Staging Layer
↓
dbt Mart Layer
↓
CSV Outputs
↓
Matplotlib Charts
```

Figure 6.2: AI Response for Dataset Understanding

6.2 Step 2: AI-Assisted Pipeline Planning

After understanding the dataset, AI was used to generate a detailed implementation plan.

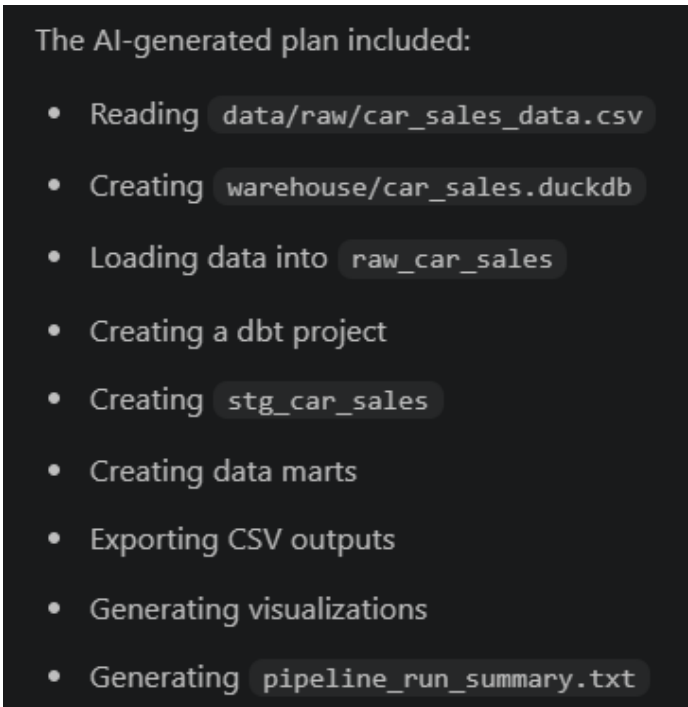
Prompt Used



```
Create a step-by-step technical pipeline plan for this car sales dataset using Python, DuckDB, and dbt.
```

Figure 6.3: Prompt for Pipeline Planning

AI Output



The AI-generated plan included:

- Reading `data/raw/car_sales_data.csv`
- Creating `warehouse/car_sales.duckdb`
- Loading data into `raw_car_sales`
- Creating a dbt project
- Creating `stg_car_sales`
- Creating data marts
- Exporting CSV outputs
- Generating visualizations
- Generating `pipeline_run_summary.txt`

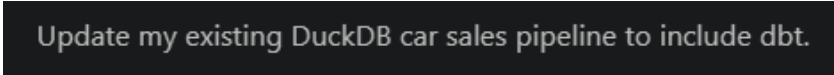
Figure 6.4: Prompt for AI-Assisted Code Generation

The generated workflow acted as a development roadmap and clearly separated the responsibilities of Python, DuckDB, and dbt. This improved project organization and reduced implementation complexity by providing a structured sequence of tasks.

6.3 Step 3: AI-Assisted Code Generation

After the implementation plan was finalized, AI was used to generate the Python code required for data ingestion and pipeline orchestration.

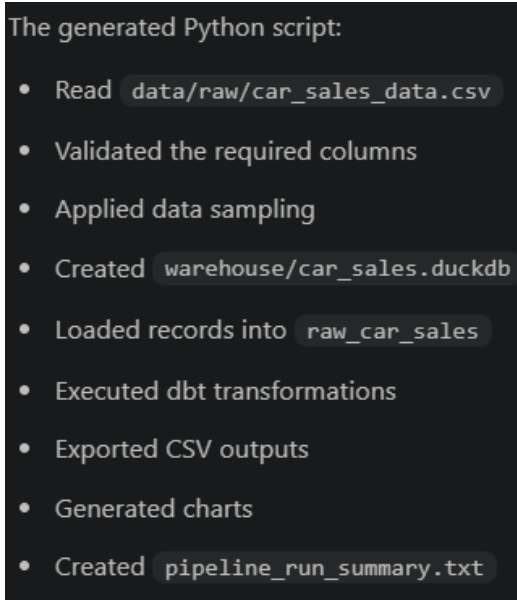
Prompt Used



```
Update my existing DuckDB car sales pipeline to include dbt.
```

Figure 6.5: Prompt for AI-Assisted Code Generation

AI Output

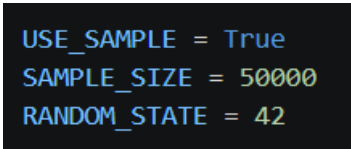


```
The generated Python script:
```

- Read `data/raw/car_sales_data.csv`
- Validated the required columns
- Applied data sampling
- Created `warehouse/car_sales.duckdb`
- Loaded records into `raw_car_sales`
- Executed dbt transformations
- Exported CSV outputs
- Generated charts
- Created `pipeline_run_summary.txt`

Figure 6.6: AI-Generated Python Data Ingestion Code

To improve execution efficiency during development, the pipeline used the following sampling configuration:



```
USE_SAMPLE = True  
SAMPLE_SIZE = 50000  
RANDOM_STATE = 42
```

Figure 6.7: Sampling Configuration Used in the Pipeline

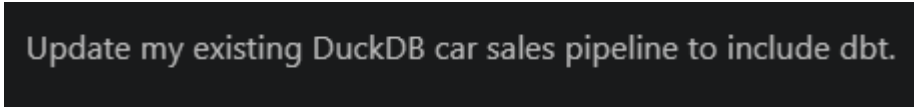
Python was selected because it provides efficient data processing through Pandas and integrates seamlessly with DuckDB. The generated code automated the ingestion process and ensured that raw data was successfully loaded into the warehouse before transformation.

The resulting `raw_car_sales` table serves as the source table for all subsequent dbt models and acts as the foundation of the data engineering pipeline.

6.4 Step 4: AI-Assisted dbt Model Generation

Following data ingestion, AI was used to generate the dbt project structure and transformation models required for the analytical layer of the pipeline.

Prompt Used



```
Update my existing DuckDB car sales pipeline to include dbt.
```

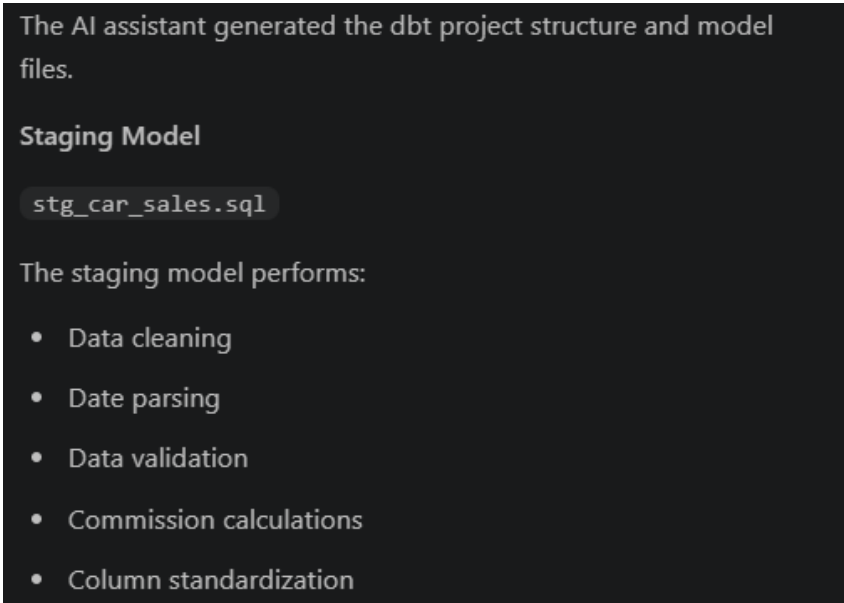
Figure 6.8: Prompt for AI-Assisted dbt Model Generation

AI Output

The AI assistant generated the dbt project structure and SQL models for both the staging and mart layers.

6.4.1 Staging Layer

The `stg_car_sales` model performs:



```
The AI assistant generated the dbt project structure and model files.
```

Staging Model

```
stg_car_sales.sql
```

The staging model performs:

- Data cleaning
- Date parsing
- Data validation
- Commission calculations
- Column standardization

Figure 6.9: AI-Generated dbt Staging Model

This staging model acts as an intermediate layer between the raw data and the analytical marts, ensuring that all downstream models use consistent and validated data.

6.4.2 Mart Layer

The following table shows the analytical marts that were generated.

Table 6.1: Analytical Mart Models Generated by dbt

Data Mart	Purpose
data_quality_summary	Evaluates overall data quality and validation results
salesperson_metrics	Analyses salesperson performance and sales contribution
car_make_summary	Summarises sales performance by vehicle brand
car_model_summary	Summarises sales performance by vehicle model
monthly_sales_summary	Analyses monthly sales trends and revenue patterns

The staging model transforms raw transactional data into a clean and standardized dataset suitable for analysis. The mart models then aggregate the transformed data into business-oriented tables that support reporting and decision-making.

Using dbt improves maintainability, scalability, and transparency by separating transformation logic from Python code. In addition, dbt automatically manages model dependencies and ensures that transformations are executed in the correct sequence.

6.4.3 Output

The successful execution of the dbt models generated the following analytical tables.

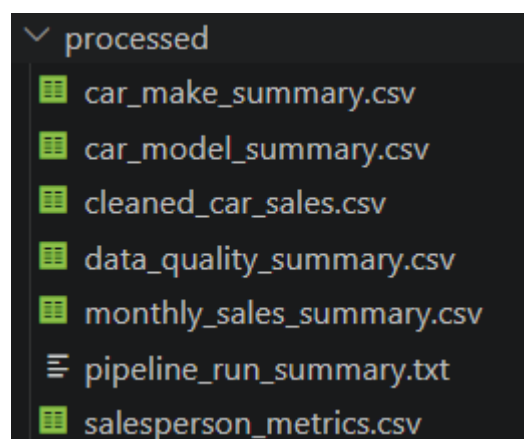


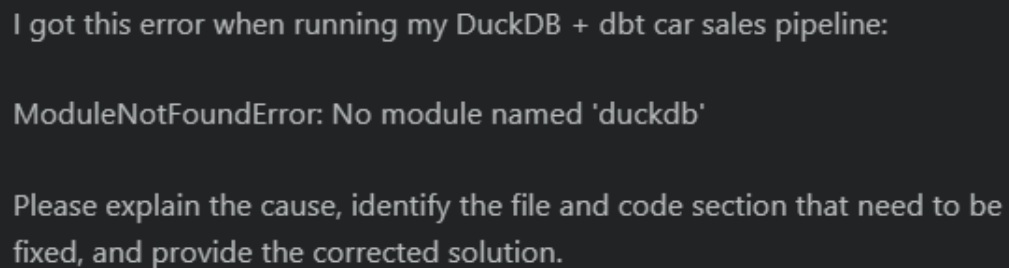
Figure 6.10: AI-Generated dbt Project Structure and Mart Models

These tables serve as the primary analytical outputs of the transformation layer and are later exported as CSV files for reporting and visualization purposes.

6.5 Step 5: AI-Assisted Debugging and Improvement

During implementation, several technical issues were encountered, including missing libraries and environment configuration problems. One example was the `ModuleNotFoundError: No module named 'duckdb'` error, which occurred because the DuckDB package was not installed in the Python environment.

Prompt Used

A screenshot of a text prompt on a dark background. The text is white and reads: "I got this error when running my DuckDB + dbt car sales pipeline: ModuleNotFoundError: No module named 'duckdb' Please explain the cause, identify the file and code section that need to be fixed, and provide the corrected solution."

I got this error when running my DuckDB + dbt car sales pipeline:

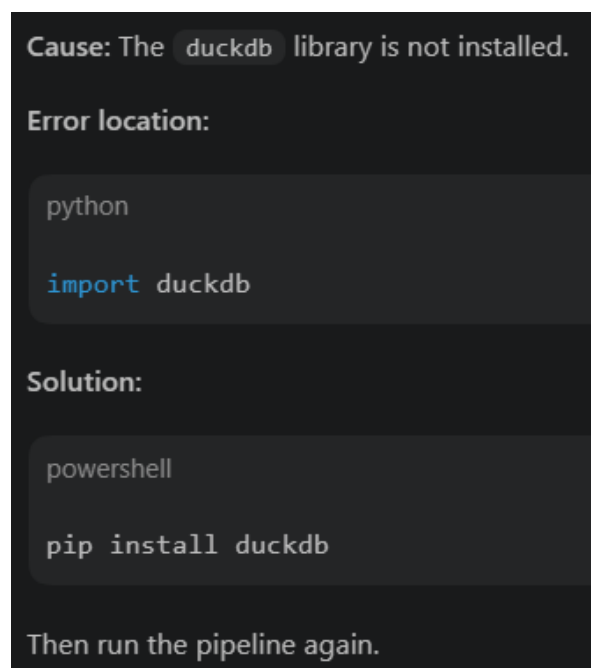
`ModuleNotFoundError: No module named 'duckdb'`

Please explain the cause, identify the file and code section that need to be fixed, and provide the corrected solution.

Figure 6.11: Prompt for AI-Assisted Debugging and Improvement

AI Output

The error message was provided to the AI assistant together with a debugging prompt. The AI assistant identified the root cause and recommended installing the required dependency using `pip install duckdb`. After the package was installed, the pipeline was executed successfully

A screenshot of AI output on a dark background. It contains three sections: 'Cause: The duckdb library is not installed.', 'Error location:' followed by a code block showing 'python' and 'import duckdb', and 'Solution:' followed by a code block showing 'powershell' and 'pip install duckdb'. It ends with the text 'Then run the pipeline again.'

Cause: The `duckdb` library is not installed.

Error location:

```
python
import duckdb
```

Solution:

```
powershell
pip install duckdb
```

Then run the pipeline again.

Figure 6.12: AI-Assisted Debugging Example

6.6 Step 6: Final Pipeline Execution

After all project files were created and the required libraries were installed, the main Python pipeline script was executed successfully. The pipeline integrated Python, DuckDB, and dbt into a single automated workflow.

The execution process consisted of the following stages:

1. Python loaded the raw CSV dataset.
2. The dataset was stored in the DuckDB warehouse as raw_car_sales.
3. Python executed the dbt run command.
4. dbt generated the staging and mart tables.
5. Python exported the transformed datasets into CSV files.
6. Python generated visualization charts using Matplotlib.
7. Python created the pipeline_run_summary.txt report.

The final outputs included the DuckDB database, processed CSV files, visualization charts, and pipeline summary report. These outputs confirmed that the end-to-end data engineering pipeline was operating as expected.

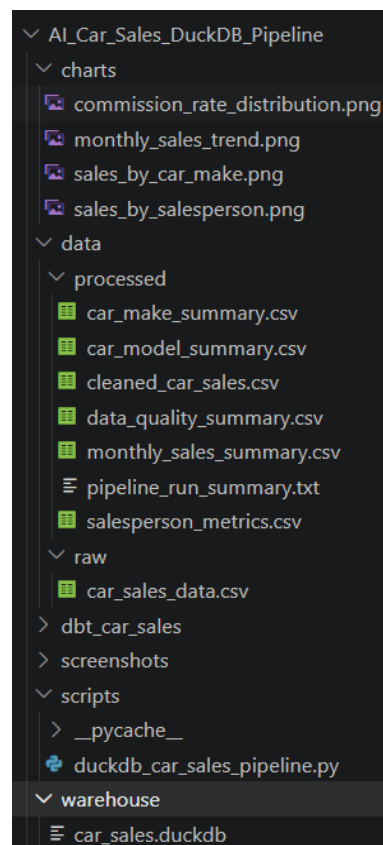


Figure 6.13: Successful Pipeline Execution Outputs

7.0 Results and Output Explanation

This section presents the outputs generated by the AI-assisted car sales data pipeline. The results demonstrate the successful use of Python, DuckDB, and dbt to transform raw sales data into structured analytical outputs. The generated outputs include DuckDB tables, CSV files, visualizations, and a pipeline summary report. These outputs provide evidence that the data ingestion, transformation, and reporting processes were completed successfully.

7.1 DuckDB Warehouse Results

The completed pipeline successfully generated multiple tables inside the DuckDB warehouse. These tables include the raw data table, staging model, and analytical marts used for data analysis and reporting. The generated tables provide a structured foundation for further analysis and support the creation of reports and visualizations.

Table 7.1: Generated DuckDB Tables

Table Name	Description
raw_car_sales	Original dataset loaded from CSV
cleaned_car_sales	Cleaned and standardized dataset
data_quality_summary	Data quality analysis
salesperson_metrics	Salesperson performance analysis
car_make_summary	Brand performance analysis
car_model_summary	Model performance analysis
monthly_sales_summary	Monthly sales trend analysis

The successful creation of these tables demonstrates that the ingestion and transformation stages were executed successfully. Raw sales transactions were transformed into structured analytical datasets suitable for reporting, performance monitoring, and business analysis.

7.2 CSV Output Results

The transformed datasets were successfully exported into CSV format to support reporting, data sharing, and further analysis. The generated outputs include cleaned sales data, data quality summaries, salesperson performance metrics, vehicle brand summaries, vehicle model summaries, monthly sales summaries, and the pipeline execution summary report.

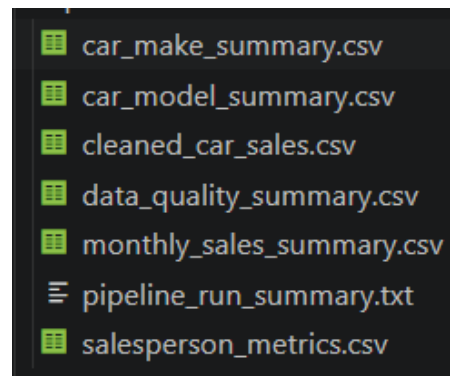


Figure 7.1: Generated CSV Outputs

The successful generation of these CSV files demonstrates that the export stage of the pipeline was completed successfully. These outputs provide portable and business-ready datasets that can be easily shared with stakeholders or integrated into reporting and visualization tools without requiring direct access to the DuckDB warehouse.

7.3 Visualization Results

The pipeline generated four visualizations using Matplotlib to support data analysis and interpretation. These visualizations provide insights into salesperson performance, vehicle brand performance, sales trends, and commission distribution.

7.3.1 Sales by Salesperson

A bar chart was generated to visualize the total sales achieved by each salesperson. The chart provides a clear comparison of sales performance across all salespersons included in the dataset. The visualization helps identify top-performing salespersons and evaluate individual sales contributions. The results can support performance assessment and provide insights into sales productivity within the organization.

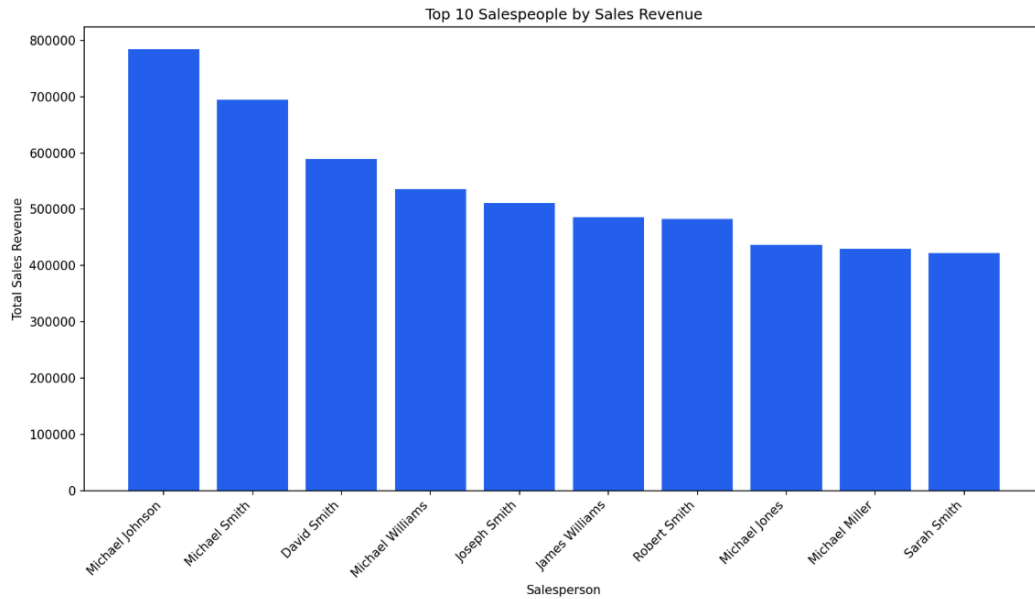


Figure 7.2: Sales by Salesperson Chart

7.3.2 Sales by Car Make

A bar chart was generated to visualize sales performance across different vehicle brands. The chart provides a comparison of total sales achieved by each car made within the dataset. The visualization helps identify the most popular vehicle brands and highlights differences in sales performance among car manufacturers. The results can support business decision-making related to product demand, sales strategies, and market analysis.

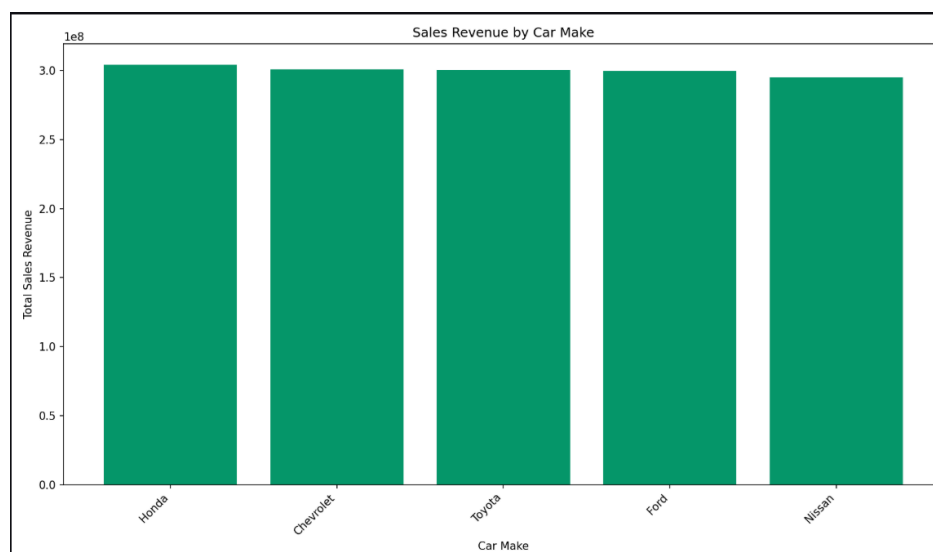


Figure 7.3: Sales by Car Make Chart

7.3.3 Monthly Sales Trend

A line chart was generated to illustrate sales trends over time. The chart provides a visual representation of changes in sales performance across different periods. The visualization reveals overall sales patterns and highlights fluctuations in performance over time. The results can support trend analysis and assist in identifying periods of strong or weak sales activity.

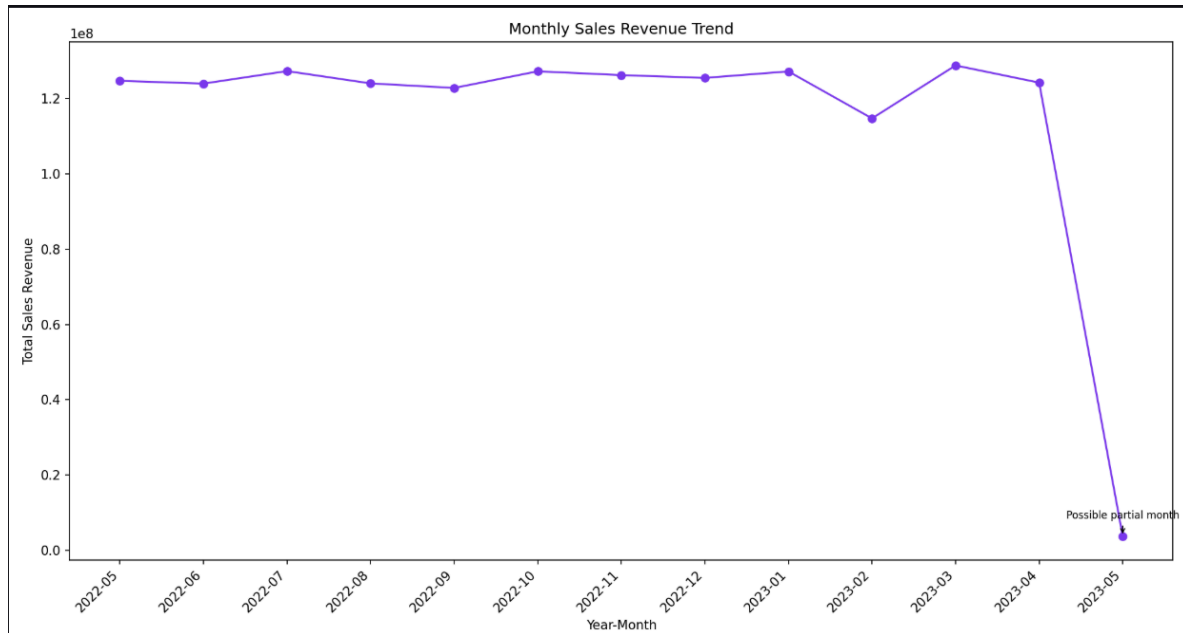


Figure 7.4: Monthly Sales Trend Chart

7.3.4 Commission Rate Distribution

A histogram was generated to visualize the distribution of commission rates within the dataset. The chart illustrates the frequency of different commission rate values. The visualization helps evaluate the consistency of commission rates and identify any unusual or extreme values. This information can support commission policy reviews and data validation activities.

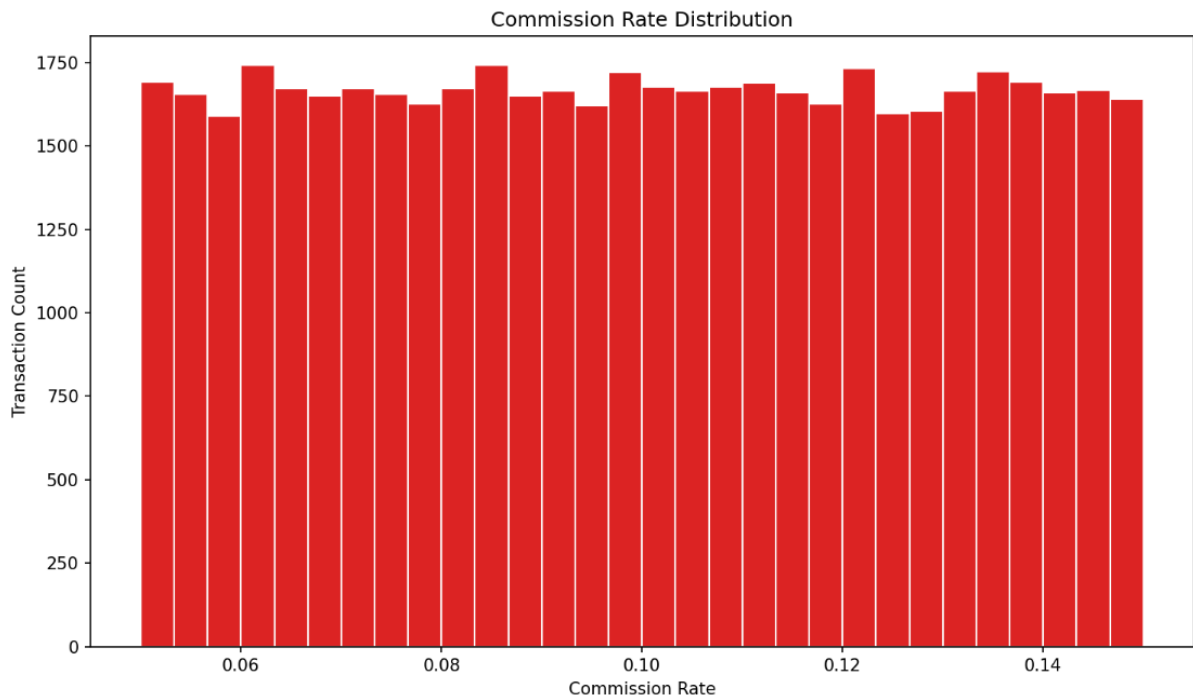


Figure 7.5: Commission Rate Distribution Chart

7.4 Pipeline Summary Report

The file `pipeline_run_summary.txt` provides a summary of the pipeline execution. The report contains dataset information, sampling configuration, row counts, generated outputs, and execution status. The summary report serves as evidence that the pipeline completed successfully and provides a concise overview of the overall execution process and generated deliverables.

```
AI Car Sales DuckDB + dbt Pipeline Run Summary

Run timestamp: 2026-06-12 23:57:50
Status: SUCCESS

Input and warehouse:
- Input CSV path: data\raw\car_sales_data.csv
- DuckDB database path: warehouse\car_sales.duckdb
- dbt project path: dbt_car_sales
- Full dataset row count: 2,500,000
- USE_SAMPLE: True
- Sample size used: 50,000
- Random state: 42

Row counts:
- Cleaned row count: 50,000
- Rows removed: 0
- Duplicate rows in sample: 0

Invalid row summary:
- Invalid dates: 0
- Invalid sale price rows: 0
- Invalid commission rate rows: 0
```

Figure 7.6: Pipeline Run Summary Report

8.0 Future Improvements

Although the implemented AI-assisted car sales data pipeline was successfully completed, several improvements can still be made to further strengthen the system. The current pipeline is able to read the raw dataset, load data into DuckDB, perform dbt transformations, export CSV files, generate charts, and produce a pipeline run summary. However, the implementation was developed mainly for tutorial and demonstration purposes. Therefore, future enhancements should focus on improving data reliability, user interaction, and pipeline automation. These improvements would make the pipeline more suitable for larger-scale usage, repeated execution, and decision-making support.

First, more comprehensive data quality checks can be added to the pipeline. In the current implementation, basic validation was performed, such as checking invalid dates, sale prices, commission rates, duplicate rows, and commission differences. In future work, additional dbt tests can be included, such as accepted value tests, range checks, uniqueness checks, and relationship checks between related fields. This improvement is important because data quality problems can affect the reliability of downstream analysis. Foidl et al. (2023) highlighted that data pipelines may produce poor-quality data due to issues such as incorrect data types, data cleaning problems, and processing errors. Therefore, stronger validation rules would improve the trustworthiness of the generated data marts.

Second, an interactive dashboard can be developed to replace or extend the current static Matplotlib charts. In this project, PNG charts were generated to show salesperson performance, car make sales, monthly sales trends, and commission rate distribution. Although these charts are useful for reporting, they do not allow users to interact with the data. In future work, a dashboard tool such as Power BI, Tableau, or Evidence can be used to provide filters by month, salesperson, car make, and car model. Research on interactive visualization has shown that visual dashboards can help users explore data and support decision-making more effectively. Therefore, an interactive dashboard would make the pipeline outputs more useful for business analysis.

Third, CI/CD automation can be introduced to improve pipeline testing and execution. In the current implementation, the pipeline was executed manually through the Python script. In future work, GitHub Actions or another CI/CD tool can be used to automatically install dependencies, run the Python pipeline, execute dbt models, perform dbt tests, and verify that

all output files are generated successfully. Shahin et al. (2017) explained that continuous integration, delivery, and deployment practices support automated testing, improve visibility of build results, and increase reliability in software delivery. Therefore, adding CI/CD would make the data pipeline more repeatable, testable, and suitable for team-based development.

Overall, these future improvements would extend the current tutorial pipeline into a more complete and reliable data engineering solution. Stronger data quality validation would improve the accuracy of analytical outputs, interactive dashboards would improve the usability of the results, and CI/CD automation would improve the repeatability of the development workflow. These enhancements would also align the project more closely with real-world data engineering practices, where data pipelines are expected to be accurate, maintainable, automated, and useful for decision-making. At the same time, human validation should continue to be applied because AI-generated code and recommendations may still require checking before being used in a production-like environment.

9.0 Reflection

LEE YIN SHEN	Through this tutorial, the process of building a data pipeline by coding with the assistance of AI was revised and strengthened. The use of AI helped in planning the pipeline structure, generating code, identifying errors, and improving the implementation. However, it was also learned that AI-generated outputs still need to be reviewed carefully to ensure that the logic and results are correct. New tools were also learned through this tutorial. DuckDB was used as a local data warehouse, where the raw, staging, and mart tables could be stored and queried using SQL. This helped improve the understanding of how a warehouse layer can be included in a data pipeline without using a cloud platform. In addition, dbt was learned as a tool for managing SQL transformations. Instead of placing all SQL statements inside one Python script, dbt allowed the transformation logic to be separated into different model files. For example, separate models were created for staging data, data quality summary, salesperson metrics, car make summary, car model summary, and monthly sales summary. This made the pipeline easier to organize, maintain, and explain. Overall, this tutorial provided practical revision on AI-assisted pipeline development while also introducing useful tools for data engineering. It showed that AI can speed up development, but the developer still needs to understand the pipeline structure, validate the outputs, and make improvements when needed.
JOANNE CHING YIN XUAN	This tutorial provided valuable exposure to AI-assisted data engineering and demonstrated how AI can support different stages of pipeline development. Through the implementation of the car sales data pipeline, a better understanding was gained of data

	ingestion, transformation, storage, and reporting using Python, DuckDB, and dbt. The tutorial also highlighted how multiple technologies can be integrated to create an end-to-end data engineering workflow. One of the key benefits observed was the ability of AI to assist with dataset understanding, pipeline planning, code generation, and debugging. AI-generated suggestions helped accelerate development and reduced the time required to solve technical issues. However, the tutorial also demonstrated that AI-generated outputs require careful validation, as generated code and recommendations may not always fully satisfy project requirements or produce the expected results. Overall, this tutorial emphasized the importance of combining AI capabilities with human judgement and technical knowledge. Future improvements could include the use of more advanced prompt engineering techniques, additional data quality validation rules, and integration with business intelligence tools for enhanced reporting and visualization. These improvements would further increase the efficiency, reliability, and scalability of AI-assisted data engineering solutions.
EVELYN GOH YUAN QI	Through this tutorial, valuable knowledge and practical experience in AI-assisted data engineering were gained. The use of AI tools demonstrated how data engineering tasks such as pipeline planning, code generation, debugging, and workflow development can be completed more efficiently. The tutorial also provided a better understanding of how AI can be integrated into modern data engineering practices to support productivity and accelerate development processes. One of the main benefits observed was the ability of AI to generate code and provide technical guidance based on natural language

	prompts. This reduced the time required to develop the data pipeline and helped resolve technical issues encountered during implementation. However, it was also learned that AI-generated outputs should not be accepted without verification, as human review is still necessary to ensure the correctness of the logic and results. Overall, this tutorial highlighted the importance of combining AI capabilities with human expertise. In future projects, more advanced prompting techniques and AI-assisted workflow automation can be explored to further improve the efficiency, reliability, and scalability of data engineering solutions.
--	---

References

- Cheng, X. (2025). Interactive data visualization for decision-making in digital government. *Proceedings of the 26th Annual International Conference on Digital Government Research*. <https://proceedings.open.tudelft.nl/DGO2025/article/view/946>
- Dwivedi, Y. K., Kshetri, N., Hughes, L., Slade, E. L., Jeyaraj, A., Kar, A. K., Baabdullah, A. M., Koohang, A., Raghavan, V., Ahuja, M., Albanna, H., Albashrawi, M., Al-Busaidi, A. S., Balakrishnan, J., Barlette, Y., Basu, S., Bose, I., Brooks, L., Buhalis, D., ... Wright, R. (2023). So what if ChatGPT wrote it? Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy. *International Journal of Information Management*, 71, Article 102642. <https://doi.org/10.1016/j.ijinfomgt.2023.102642>
- Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2023). Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *arXiv*. <https://arxiv.org/abs/2309.07067>
- Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5, 3909–3943. <https://doi.org/10.1109/ACCESS.2017.2685629>
- Zhang, Q., Fang, C., Xie, Y., Zhang, Y., Yang, Y., Sun, W., Yu, S., & Chen, Z. (2024). A survey on large language models for software engineering. *arXiv*. <https://arxiv.org/pdf/2312.15223>